

CareerOS

User problem first, then expand the interface.

What does it actually take to turn five separate, manual job-application tasks into one system a candidate can trust? *CareerOS is a five-step AI workflow that carries a candidate from a raw job posting to interview-ready, in one continuous session. Job analysis feeds resume optimization, which feeds the cover letter and application email, which feeds interview preparation, which feeds interview coaching — each step consuming the output of the one before it, all within one persisted session.*

Role	Product Builder — end-to-end: design, development, deployment
Project type	AI Product / Workflow Automation
Status	V5.3 released — core workflow complete and running end-to-end
Scope	8 iterative build cycles, from job analysis to session persistence
Focus areas	AI workflow orchestration · Session state design · MVP scoping
Stack	Next.js · TypeScript · Tailwind CSS · Gemini 2.5 · Airtable · Vercel · GitHub

THE PROBLEM

Business Context

Every job application is a self-contained unit of work: a job posting, a resume, and eventually an interview. Before CareerOS, a candidate reruns the same manual sequence for each one — read the job description for keywords, adjust the resume by hand, draft a cover letter and application email from scratch, guess likely interview questions, and rehearse answers with no structured feedback. Nothing produced in one step carries into the next, and nothing from one application session survives once the tab closes.

Stakeholders

In a real deployment, this platform would serve three groups, each asking a different question of the same session:

- **Candidate** — Can I pick up where I left off, and revisit coaching whenever I need to?
- **Recruiter / ATS** (indirect — sets the bar the outputs are judged against) — Does this resume and cover letter actually match what we're screening for?

- **Product owner** (session data) — Is the AI behaving reliably in production — where does it fail, and do guardrails and fallbacks catch it?

Problem Statement

Job applications don't build on each other — each one starts from zero.

Objectives

- **Carry the candidate through the whole application.** All five stages, without re-entering context at each step.
- **Persist every session.** Progress isn't lost between visits.
- **Produce outputs a recruiter would recognize as tailored,** not generic or templated.
- **Support the language the job posting is actually written in** — French, English, or Spanish.

What "success" means here. CareerOS is a real, deployed product, not a simulated dataset. Every step has been run against my own resume and real job postings — not synthetic test data — but it hasn't been tested with other users yet, so nothing here should imply validated outcomes for a broader audience. Success is measured differently, and more honestly: does every step produce output a recruiter would recognize as genuinely tailored? does session state persist correctly through all five stages, in three languages, without silently dropping context? does the architecture hold at each version instead of accumulating undocumented shortcuts?

PRODUCT THINKING

Scope — in

- A 5-step guided workflow: Job Analysis → Resume Optimization → Cover Letter & Email → Interview Prep → Interview Coaching
- Resume input via Word file upload or pasted text
- Session persistence via Airtable, tracked across all 5 stages

Scope — out, and why

- **PDF resume upload.** Dropped after a parsing bug; Word upload and pasted text cover the same input need without the fragility.
- **Job posting URL import.** Deferred for MVP; pasting the posting covers the core loop without building a scraper for arbitrary job sites.
- **PDF export of outputs.** Deferred for MVP; finishing the 5-step flow and session persistence took priority over adding a delivery format.
- **User accounts / authentication.** Deferred; a single-session model was enough to validate the workflow before building multi-user infrastructure.

Understanding the workflow

The system has exactly one generative boundary — everything before it is capturing input, everything after it is persisting and letting the candidate resume:

DETERMINISTIC – INPUT & STATE

Job posting (paste) → Resume (Word upload / paste) → Session created

↓

GENERATIVE – GEMINI, FIVE STEPS IN SEQUENCE

Job Analysis → Resume Optimization → Cover Letter & Email → Interview Prep → Interview Coaching

↓

DELIVERY – PERSISTED & RESUMABLE

Session saved to Airtable → candidate can leave and resume at any stage

SOLUTION & DECISIONS

MVP: what is the least this can do and still be useful?

The essential line stops at a working resume, cover letter, interview prep package, and coaching feedback — produced in one continuous, persisted session. A candidate who completes the five steps leaves with a usable application and interview readiness, without touching another tool. File parsing, URL scraping, export formats, and accounts are real needs, but none of them change whether the candidate leaves with something usable. That's why they're the first things cut when the line had to be drawn.

Solution Design

Built in sequence: input handling first, then the five-step Gemini chain wired step-to-step, then Airtable session persistence layered on top once the chain worked end to end.

The layers. Each has one job, and consumes only what the layer before it published:

- **Input layer** — captures the job posting (pasted text) and the resume (Word upload or pasted text), normalized into session state.
- **Generation layer** — five API routes (/api/analyze, /api/optimize-resume, /api/generate-application, /api/interview-prep, /api/interview-coach), each calling Gemini with the accumulated session state available to it.
- **Persistence layer** — each completed step writes into an Airtable session record, so state outlives the browser tab.
- **Delivery layer** — a single-page frontend indexed by currentStep, moving the candidate through completed steps with Previous/Next.

The rule that governs the whole design: **one session state, additive throughout the workflow.** Every step writes its output into the same state object — jobOffer, resume, jobAnalysis, resumeOutput, applicationOutput, interviewOutput, coachFeedback — allowing subsequent steps to reuse previously

generated data without asking the user to re-enter it. The accumulated state is cleared only when a new session is explicitly started.

AI Design Considerations

AI Role

Job Analysis	Extracts structured signal from the posting — skills, requirements, ATS keywords, tone, strengths, summary.
Resume Optimizer	Rewrites and optimizes the resume using only the candidate's existing information.
Cover Letter & Email	Generates a professional application email and cover letter adapted to the offer.
Interview Prep	Generates likely interview questions and answer frameworks.
Interview Coach	Evaluates a candidate's draft answer and suggests improvements, without altering their real experience.

Every prompt instructs the model never to invent experience, achievements, companies or skills. The model generates content only from the candidate's input; it doesn't classify candidates, retrieve external data, use tools or retain memory across sessions.

Evaluation

Evaluation today is workflow-based rather than benchmark-based. Outputs are reviewed manually for structural correctness, language consistency, factual fidelity to the candidate's input, and successful consumption by the next stage. Formal prompt evaluation remains future work.

Failure Modes

- **Quota or availability** — a `429 RESOURCE_EXHAUSTED` or `503` from Gemini drops the app into Demo Mode.
- **Output format drift** — prompt changes can break downstream consumers. This happened during V5.3, when translated structural labels prevented Interview Coach from parsing Interview Prep correctly (see Reflection).
- **Oversized uploads** — not an AI failure but a workflow one: Vercel returns `413 Payload Too Large` for large DOCX files.

Almost nothing fails silently — a Gemini failure surfaces as Demo Mode, and a broken step surfaces immediately in the one that follows it.

Guardrails

- **Prompt guardrails** — never invent experience, achievements, companies, or skills; use only the provided information; no extra sections; follow the exact requested structure.
- **Language guardrails** — detect the job posting's language automatically.

- **Structural guardrails** — every prompt forces a numbered section format (### 1, ### 2...).
- **Technical guardrails** — in Interview Prep, the parser-critical labels (Question:, Suggested Answer Framework:, Context:, Action:, Result:) stay in English regardless of output language.

Fallback Strategy

When Gemini doesn't respond — quota exhausted, timeout, 503, any API error — every endpoint returns a Demo response specific to that step instead of failing the request (DEMO MODE – Gemini quota unavailable.).

There's no automatic retry, no provider switching, no model fallback, no response caching. Demo Mode is a safe stop, not a resilience system — and the roadmap treats it as one.

Key Decisions

01

Airtable to persist session data, instead of a custom database.

It provided a structured, queryable store with minimal implementation overhead, allowing the session lifecycle to be implemented without adding database infrastructure.

02

Session-stage completion as the evaluation signal, not a formal eval harness.

Rather than introducing a formal evaluation framework at this stage, I treated successful progression through the workflow as the primary quality signal. A session reaching Interview Coaching indicates that each previous output was usable enough for the system to continue.

03

Automatic language adaptation based on the job posting.

Outputs automatically follow the language of the uploaded job posting. This keeps every stage of the workflow consistent without requiring additional user input.

RESULTS

Tailored output. Validated manually against real job postings and my own resume.

End-to-end workflow completed. All five stages function as one integrated system, from job posting to interview coaching, inside a single session.

Persistent session state. Context is preserved throughout the workflow — the same Airtable record is created, updated, and evolves from the first step through Coaching Completed.

Multilingual workflow. English, French, and Spanish are supported by the same workflow and the same prompts.

Architectural stability. Successive features — multilingual support, session persistence, Interview Coach — were integrated without redesigning the overall workflow or breaking the session model.

RISKS & ASSUMPTIONS

- **Prompt-only guardrails aren't enforced.** "Never invent experience" is an instruction to the model, not a validated constraint — nothing downstream checks that it was actually followed.
- **No output-format validation.** The same class of failure that broke Interview Prep's parser in French (see Reflection) could resurface with any future prompt or model change, since nothing validates structure before it's consumed by the next step.
- **No resilience beyond Demo Mode.** During a real Gemini outage, the product can only offer demo content, not real output, until Gemini recovers — there's no retry and no alternate provider.
- **Single point of AI dependency.** The entire workflow runs on Gemini; a provider-side change — pricing, deprecation, rate limits — affects every step, not just one.
- **No account-based session recovery.** Sessions are persisted in Airtable, but there is no account-based mechanism to retrieve them later. Without authentication or a session retrieval flow, users cannot reliably resume previous work across devices or browser sessions.

WHAT I'D DO NEXT

- **Session History and resumable sessions** — session retrieval API, history page, read-only session viewer.
- **User accounts and multi-user architecture** — unlocks session recovery by identity, not just by browser tab.
- **Real output validation** — a schema checked before an output reaches the next step, so format drift is caught before it breaks a parser, not after.
- **A lightweight evaluation harness** — even a small set of postings and resumes run against each prompt version, to catch regressions before they reach a real session.
- **Resilience beyond Demo Mode** — retry logic and a provider fallback, so a Gemini outage degrades gracefully instead of switching every step to demo content.
- **PDF resume upload, job URL import, and PDF export** — real candidates for expansion once the core workflow and its reliability are proven.

None of these change the architecture. They extend it — which is the point of having drawn the boundaries where I did.

REFLECTION

Interview Prep would finish, but Interview Coach would open to "Question 1 of 0" — no questions to coach on, on the one step meant to close the loop. The cause wasn't the content. Gemini had translated the job posting's language correctly, but it had also translated the structural labels the parser depended on — "Question:" became something else, and the parser, looking for the literal English label, found nothing.

I'd been treating "respond in the job posting's language" as one instruction. It needed to be two: respond in that language, and keep these specific labels in English regardless of it. I chose to lock the technical labels in the prompt rather than making the parser progressively more tolerant. The ambiguity originated in the prompt, so that was the most appropriate place to remove it.

The question was what it takes to turn five separate tasks into one system a candidate can trust. This is the answer in miniature: trust isn't just the model producing good content — it's the system staying legible to itself once the model behaves differently from the assumptions the rest of the system was built on.

Sabrina Urriola · Product Builder | AI & Automation · CareerOS · Full case study